

《基于 Transformer 的字符级多位数加法实验》

作业名称： 基于 Transformer 的字符级多位数加法实现

目录

1. 作业背景
2. 作业目标
3. 作业任务概述
4. 数据集构建
5. 模型结构 (Transformer Decoder-only)
6. 作业步骤
7. 作业结果展示 (loss 曲线 + 样例输出)
8. 思考题
9. Python 程序代码

1. 作业背景

Transformer 是当前深度学习、大语言模型 (LLM) 中最重要的模型结构之一。本作业旨在通过一个**可控、可解释、可验证**的任务——**字符级多位数加法**，实现一个小型 Transformer，并观察其学习过程。

作业任务示例：

输入： "123+456="

输出： "579"

作业思路提示：把整个算式视为一个**字符级语言模型 (Character-level LM)** 任务，让 Transformer 模型自回归预测下一个字符。

2. 作业目标

通过本作业，学生将能够使用 Python 完成以下任务：

- 理解 Transformer Decoder 的结构与计算流程
- 掌握字符级语言模型的训练方法
- 能够构建可控数据集并训练模型
- 绘制训练/验证 loss 曲线
- 撰写作业报告并提供 Python 程序代码

3. 作业任务概述

本作业包含以下部分：

1. **数据生成**：自动生成多位数加法样本
2. **模型实现**：从零实现一个 Tiny Transformer (Decoder-only)
3. **训练与验证**：绘制 loss 曲线
4. **结果分析**：观察模型输出与错误模式

4. 数据集构建

4.1 字符表

包含：

- 数字：0-9
- 符号：+, =

- 空格：用于 padding

4.2 样本生成

每个样本格式：

$a+b=$ 结果

例如：

"12+7=19"

4.3 数据集规模（可自定）

- 训练集：20,000 条
- 验证集：2,000 条
- 最大序列长度：16

5. 模型结构 (Transformer Decoder-only)

模型结构如下：

1. Token Embedding
2. Positional Encoding
3. N 层 Transformer Block
 - Multi-Head Self-Attention
 - Feed Forward Network
 - Residual + LayerNorm
4. 输出层 (Linear $\rightarrow$ vocab_size)

注：这是一个简化版 GPT 模型。

6. 作业步骤（可自定）

Step 1: 准备数据

运行 `make_dataset()` 函数生成训练/验证集。

Step 2: 构建模型

使用 `TinyTransformerLM` 类。

Step 3: 训练模型

记录每个 epoch 的 train loss 和 val loss。

Step 4: 绘制 loss 曲线

使用 `matplotlib` 绘制：

- train loss 曲线
- val loss 曲线

Step 5: 测试模型

输入 "123+456=", 让模型自回归生成结果。

7. 作业结果展示

学生需提交：

7.1 Loss 曲线图

- train loss
- val loss

7.2 模型输出示例

例如：

输入： "123+456="

模型输出: "579"

8. 思考题 (自己思考, 无须回答)

1. 为什么 Transformer 必须使用位置编码?
2. 为什么残差连接对深层网络训练至关重要?
3. 多头注意力相比单头注意力的优势是什么?
4. 为什么模型容量不足时无法学习更复杂的加法?
5. 如果把任务改成乘法, 模型能学会吗? 为什么?

9. 提供 Python 程序代码